

Performance Analysis of State Space Models for Visual Odometry

Ashvin Anilkumar

Department of Electrical and Computer Engineering
University of Wisconsin-Madison, United States
anilkumar4@wisc.edu

Abstract: State Space Models have $O(N)$ inference complexity and competitive representational capacity, but how they behave inside a complete, deployed robotics pipeline has not been studied. We present the first evaluation of MambaGlue — an SSM-based visual feature matcher — within a full monocular visual odometry stack in Gazebo Fortress, benchmarked against SuperGlue, LightGlue, and LightGlue with adaptive point pruning on trajectory accuracy and GPU utilization. MambaGlue achieves the lowest Absolute Pose Error of all four methods (RMSE 3.380 m; 4.75% of path length), outperforming the next-best transformer-based method by 5.6%, while averaging $\approx 61\%$ GPU utilization versus $\approx 66\text{--}67\%$ for the attention-based matchers, which periodically saturate the device. Profiling reveals that SuperPoint accounts for 67–79% of per-frame inference time across all configurations; the latency spread attributable to matcher choice is only 12.3 ms in total. Replacing the matcher will not get this pipeline to real-time — replacing the detector will. Source code is available at [1]

Keywords: Mamba, State Space models, Visual Odometry, Robotics

1 Introduction

The way in which a robot analyses and understands its surroundings is a pivotal factor that determines how useful a robot is in any scenario. Therefore, various sensors, techniques, and systems for mobile robot positioning, such as wheel odometry, laser/ultrasonic odometry, global position system (GPS), global navigation satellite system (GNSS), inertial navigation system (INS), and visual odometry (VO) have been developed in the past couple of years[2]. This process needs to be both accurate and fast, and it’s still an open challenge. We decided to take on visual odometry since it tests the limits of the inference pipeline to generate real-time data to its limits.

State Space Models (SSMs), and specifically the Mamba S6 architecture, have emerged as a promising alternative. They promise Transformer-level representational capacity with linear $O(N)$ inference complexity and a significantly smaller GPU memory footprint. Recent works in building applications using state space models include MambaGlue[3], which is the first published application of an SSM layer inside a visual feature matcher, replacing the $O(N^2)$ self-attention block in SuperGlue with a selective scan. However, MambaGlue’s performance has only been validated on standard offline benchmarks — not inside a live robotics stack.

This paper addresses that gap with three concrete contributions:

- **First live-pipeline evaluation of SSM-based feature matching for visual odometry.** We deploy MambaGlue inside a complete ROS 2 monocular VO stack and benchmark it head-to-head against SuperGlue, LightGlue, and LightGlue-Adaptive on a controlled simulated trajectory, yielding the first comparison of SSM and attention-based matchers in an end-to-end robotics context.

- **MambaGlue achieves best-in-class trajectory accuracy with lower GPU utilization.** Across a 71.2 m indoor trajectory, MambaGlue achieves the lowest APE RMSE (3.380 m; 4.75% relative to path length) of all four evaluated methods while averaging $\approx 61\%$ GPU utilization — demonstrating that linear-complexity SSM matching does not sacrifice geometric accuracy and imposes less resource pressure than transformer-based alternatives.
- **Identification of the keypoint detector as the sole real-time bottleneck.** Component-level timing shows that SuperPoint alone consumes 67–79% of the per-frame inference budget across every configuration tested. The matcher contributes at most 12.3 ms to the total latency spread, placing the path to real-time VO entirely in the detector stage. This diagnostic finding directly motivates replacing SuperPoint with lightweight alternatives — such as DISK [4] or XFeat (Potje et al., 2024) [5], which achieves comparable accuracy to SuperPoint at a fraction of the computational cost — as the single highest-leverage step toward real-time SSM-based visual odometry.

2 Background

State Space Models (SSMs) provide a robust mathematical framework for sequence modeling by updating a hidden state representation, h_t , through a system of state and output equations governed by learned transition matrices (A , B , C , and D). While foundational SSMs operate as continuous-time, Linear Time-Invariant (LTI) systems requiring discretizations via a step-size parameter (Δ) for digital implementation, they inherently struggle with contextual filtering due to their static parameters. Modern architectures, notably the Mamba framework, address this limitation by introducing a structured selective mechanism wherein the parameters B , C , and Δ are explicitly parameterized as functions of the input x_t .

MambaVO [6] (CVPR 2025) is the closest prior work: it applies a Mamba-based sequential matching refinement module to deep visual odometry, capturing inter-frame temporal dependencies at $O(N)$ cost. Unlike MambaVO, which integrates the SSM into an end-to-end learned pose estimator, our pipeline treats the SSM purely as a drop-in matcher within a classical Essential Matrix front-end, enabling a controlled comparison against transformer-based and GNN-based matching alternatives.

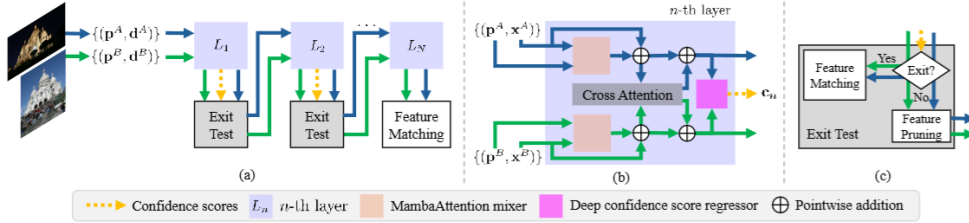


Figure 1: An overview of the MambaGlue pipeline, from [3]. The image depicts the keypoint and descriptor extraction from two frames. The keypoints are passed on to Mamba layers, in which the state space models reside.

Monocular VO is subject to several compounding failure modes: scale ambiguity (metric translation is unrecoverable without additional priors), drift from open-loop pose integration, degeneracy under pure rotation, and sensitivity to low-texture environments. A further tension is computational — transformer-based matchers such as SuperGlue achieve high accuracy via $O(N^2)$ self-attention, but that cost grows quadratically with keypoint count and can exceed real-time budgets on constrained hardware.

Through this experiment, we characterize the accuracy and hardware utilization of MambaGlue’s $O(N)$ SSM-based matching head within a live ROS 2 pipeline, and measure the per-component latency distribution to determine which stage of the pipeline limits throughput. We handle scale ambiguity at evaluation time via `evo_ape’s correct_scale` flag, which is standard practice for

monocular VO evaluation; we mitigate pose degeneracy through RANSAC [7] inlier thresholding and a ground-plane constraint that zeroes out physically impossible Z-axis motion for a differential-drive robot; and we address the texture-sensitivity challenge by designing the Gazebo simulation world with deliberate surface variety — brick walls and concrete ground — ensuring SuperPoint [8] has sufficient gradient structure to produce the keypoint density required for reliable matching across all four configurations.

3 Methodology

Since we are resource-constrained, we conduct this experiment in a Gazebo simulation, where a differential-drive robot equipped with a monocular camera navigates the environment at 30 Hz. The captured frames are stored sequentially and then processed offline through our visual odometry pipeline.

The first stage of the pipeline is feature extraction. For each frame, we use SuperPoint to detect keypoints and compute their associated descriptors. We prefer SuperPoint over classical alternatives such as ORB[9] or SIFT[10] because MambaGlue — the matcher used in the next stage — was trained specifically on SuperPoint descriptors, making the two components well-suited to one another.

The keypoints and descriptors from two consecutive frames are then passed to MambaGlue, which produces a set of confident cross-frame correspondences. From these point matches, we estimate the essential matrix between the two views using a RANSAC-based solver, which simultaneously filters outlier matches. Since our camera parameters are known from the Gazebo model, we can work directly in calibrated image space. The essential matrix is then decomposed via SVD — using `recoverPose` — to recover the relative rotation R and translation t between the two frames. These relative poses are accumulated incrementally to reconstruct the full estimated trajectory.

We perform this experiment on 4 inference pipelines – MambaGlue[3], SuperGlue[11], LightGlue[12], and LightGlue with its adaptive mode turned on. We include SuperGlue and LightGlue as baselines, as both are transformer-based implementations and will give us a clear idea of how much improvement the SSM-based approach brings to the table. We perform a benchmarking on the time taken by each stage of the pipeline and do a brief analysis of the maximum FPS that is achieved using our odometry pipeline. We will also analyze the accuracy of the predicted trajectory using official standards such as Absolute Trajectory Error.

4 Experimental Design

4.1 Simulation Environment

All experiments are conducted in Gazebo Fortress, chosen for its deterministic, reproducible sensor simulation. The robot is a differential-drive platform with a front-facing monocular RGB camera at 30 Hz; ground-truth pose is obtained from the `/odom` topic via Gazebo’s robot-state plugin. The simulated world is a textured indoor scene with rich surface detail — a deliberate choice to ensure sufficient SuperPoint detections and isolate matcher performance from scene-level confounds.

4.2 Data Collection Protocol

A single continuous sequence covering both translational and mild rotational maneuvers is collected via teleoperation. All computational benchmarks are measured by replaying this fixed frame set offline rather than against a live sensor stream; this eliminates operator-induced variance in robot speed and ensures that latency measurements are reproducible across runs.

4.3 Inference Pipeline

The pipeline follows the design described in Section 3: SuperPoint detection $\rightarrow$ matcher $\rightarrow$ RANSAC Essential Matrix (`findEssentialMat`) $\rightarrow$ pose recovery (`recoverPose`). The only variable across the four experimental conditions is the matching module; all other stages are held fixed.

4.4 Degeneracy Handling and Trajectory Accumulation

Frame pairs with fewer than 20 RANSAC inliers are classified as degenerate; the pipeline holds the last valid pose rather than propagating an unreliable estimate. For well-conditioned frames, the relative pose (R_t, t_t) is composed with the accumulated global pose via standard dead-reckoning. The recovered translation is a unit vector, so metric scale is unavailable and is corrected at evaluation time via Sim(3) alignment.

5 Experimental Results

5.1 Experimental Setup

All experiments were conducted on an NVIDIA RTX 4090 running Ubuntu 22.04 with ROS 2 Humble, against a 71.2 m indoor trajectory (290.7 seconds, 3,414 ground-truth poses) logged in TUM format. Four matching backends were evaluated — MambaGlue [3], LightGlue [12], LightGlue-Adap [12], and SuperGlue [11] — all sharing the same SuperPoint [8] detector, a confidence threshold of 0.5, and the same RANSAC pose estimation stage; only the matching module differs. Latency statistics were collected over 500 frame pairs with the first 50 excluded to avoid CUDA JIT artefacts; APE was computed with the `evo` package using Sim(3) Umeyama alignment and scale correction.

5.2 Computational Performance

Method	Wall (ms)	Std (ms)	P95 (ms)	FPS	SP (ms)	Matcher (ms)	Geo (ms)	GPU Util.	Peak
MambaGlue	128.2	26.9	173.8	7.80	92.6	31.9	3.6	61.0%	91.0%
LightGlue	115.9	27.6	159.5	8.63	90.0	22.3	3.5	66.0%	100.0%
LightGlue-Adap	118.1	21.1	153.3	8.47	93.8	20.4	3.8	60.0%	88.0%
SuperGlue	121.3	28.2	176.3	8.24	86.3	31.6	3.3	66.6%	100.0%

Table 1: Inference latency and GPU resource usage (500 frame pairs, RTX 4090).

MambaGlue records the highest mean wall time at 128.2 ms (7.80 FPS), with LightGlue the fastest at 115.9 ms (8.63 FPS); the difference traces almost entirely to the matcher stage (31.9 ms vs. 22.3 ms). Across all configurations, SuperPoint accounts for 67–79% of total inference time, making it the dominant bottleneck; the geometry stage adds a negligible 3.3–3.8 ms regardless of matcher choice. All four methods operate well below the 25 FPS real-time threshold, confirming that any path to real-time VO must first address the detector rather than the matcher.

LightGlue-Adaptive achieves the most consistent latency ($\sigma = 21.1$ ms vs. 26.9–28.2 ms for the others) and the lowest P95 of 153.3 ms, reflecting its point pruning mechanism. SuperGlue and MambaGlue impose comparable matcher costs (31.6 ms and 31.9 ms respectively). No frame pairs across any method were flagged as degenerate, confirming consistent feature coverage throughout the trajectory.

5.3 Trajectory Accuracy

Table 2 reports APE statistics after Sim(3) alignment and scale correction; scale factors were consistent in magnitude across all four methods, confirming a common footing for comparison.

MambaGlue produces the lowest APE RMSE at 3.380 m (4.75% of path length), ranking ahead of LightGlue-Adaptive by 5.6%, SuperGlue by 9.3%, and LightGlue by 9.9%, with consistent ordering across mean, median, and standard deviation. The absolute RMSE differences (0.20–0.37 m)

Method	RMSE (m)	Mean (m)	Median (m)	Std (m)	Min (m)	Max (m)	Rel. RMSE (%)
MambaGlue	3.380	3.093	3.151	1.363	0.630	6.329	4.75
LightGlue-Adap	3.581	3.312	3.198	1.361	0.159	7.538	5.03
SuperGlue	3.726	3.436	3.729	1.443	0.056	5.575	5.23
LightGlue	3.750	3.434	3.399	1.508	0.067	5.784	5.27

Table 2: Absolute Pose Error (APE) — translation component, Sim(3) aligned and scale-corrected.

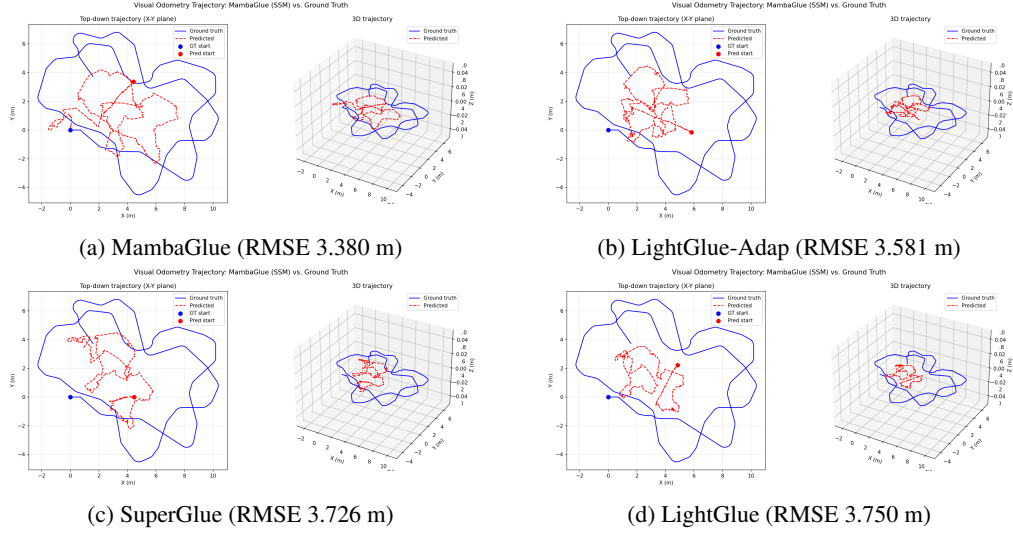


Figure 2: Trajectory overlays (estimated vs. ground truth) after Sim(3) alignment and scale correction. All four methods recover the general shape of the ground truth path; drift accumulates in the latter half of each run.

should be treated as indicative rather than conclusive given the single-trajectory, stochastic evaluation. LightGlue-Adaptive ranks second in both accuracy and latency consistency, suggesting its match pruning removes noisy correspondences that would otherwise propagate into pose estimates. SuperGlue and LightGlue rank third and fourth with nearly identical RMSE values (3.726 m and 3.750 m).

Trajectory overlays and GPU utilisation profiles are shown in Figures 2 and 3. MambaGlue’s error ceiling (6.329 m) is tighter than LightGlue-Adaptive’s (7.538 m), though its minimum APE (0.630 m) is the highest of any method, reflecting a less accurate estimate in the early stationary segment. Drift accumulates for all methods in the latter half of the trajectory, consistent with the open-loop integration design.

6 Discussion and Limitations

6.1 Real-Time Gap and the Cost–Accuracy Trade-off

A first-order finding of this evaluation is that none of the four configurations reaches real-time performance: all operate in the 7.80–8.63 FPS range against a 25 FPS target. This is not a marginal shortfall — it represents a factor-of-three gap — and it is entirely attributable to SuperPoint rather than to any of the matchers (Section 6.2). This result should be understood as the headline computational outcome of this study.

Within that shared sub-real-time operating regime, MambaGlue produces the lowest APE RMSE on this trajectory, at the cost of roughly 8–12 ms of additional per-frame latency compared to the fastest configuration. That trade-off may be acceptable in applications where trajectory fidelity matters more than raw throughput — for instance, in autonomous inspection or slow-speed mapping tasks

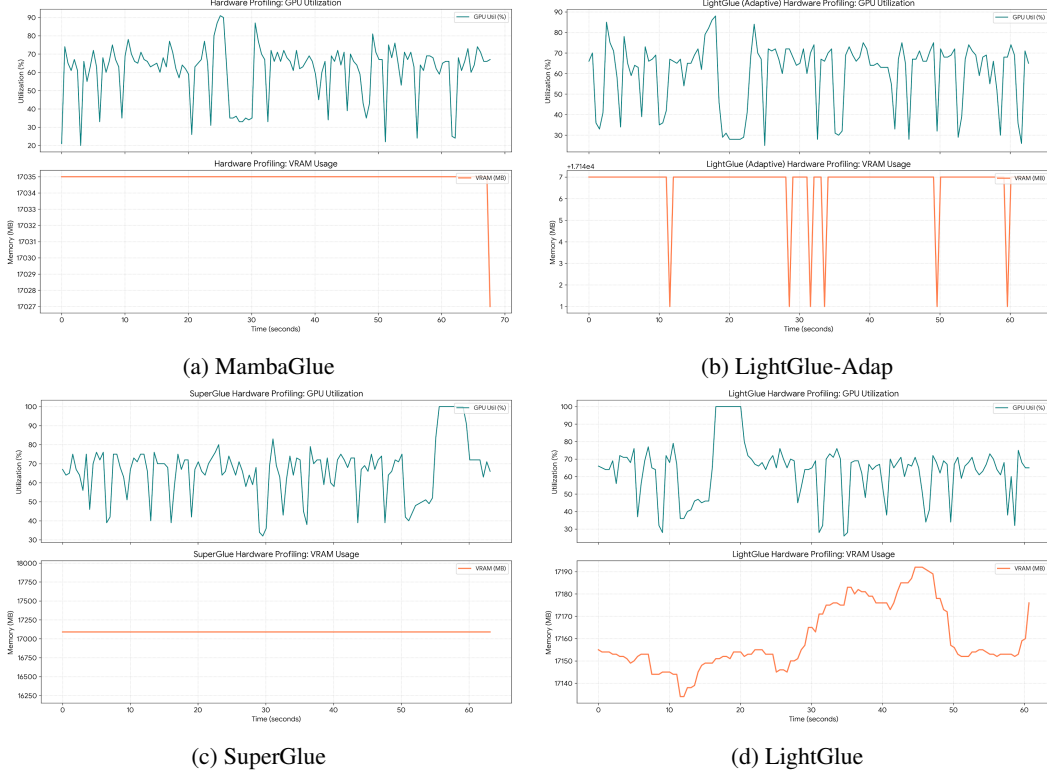


Figure 3: GPU utilisation (%) sampled every 500 ms over the 500-frame benchmark run for each method. MambaGlue and LightGlue-Adap average ≈ 60 – 61% ; LightGlue and SuperGlue average ≈ 66 – 67% and occasionally saturate the device.

— but the underlying pipeline would need to be substantially accelerated before either consideration applies to real-time deployment.

One possible explanation for MambaGlue’s accuracy advantage is that SSMs model keypoint correspondences sequentially, carrying context forward through a recurrent-like state, which may produce match sets with fewer geometrically implausible outliers that would otherwise survive RANSAC filtering. This remains a hypothesis: match-level evidence (inlier ratios, match distributions) was not collected in this work, and the margin between methods on a single trajectory is insufficient to distinguish a genuine architectural effect from trajectory-specific or stochastic variation. Whether SSM-based matching confers a systematic accuracy benefit across diverse environments is an open question that warrants controlled follow-up experiments.

6.2 SuperPoint is the True Bottleneck

Despite the focus on matching architectures, the component timing data makes one thing clear: SuperPoint consumes 67–79% of the total inference budget across all configurations. The matching modules differ by up to 11.5 ms (20.4 ms for LightGlue-Adap versus 31.9 ms for MambaGlue), but the overall latency spread across methods is only 12.3 ms precisely because SuperPoint sets a shared floor. Any serious attempt to achieve real-time performance with this pipeline would need to address the detector first — whether by replacing SuperPoint with a lighter keypoint extractor, reducing the number of detected points, or running detection and matching in a pipelined fashion across consecutive frames.

6.3 GPU Utilisation and Framework Overhead

The reported VRAM usage for all methods (≈ 17 GB) reflects total GPU memory as observed by `nvidia-ml-py3`, which includes the PyTorch CUDA runtime context, CUDA kernel caches, and any other processes active on the GPU. The inter-method variation is small (under 150 MB), confirming that the incremental VRAM footprint of swapping matchers is negligible. MambaGlue and LightGlue-Adap both average around 60–61% GPU utilisation, while LightGlue and SuperGlue average 66–67% and occasionally saturate the GPU at 100% (Figure 3), suggesting that the SSM-based and adaptive pruning approaches impose somewhat lower average GPU load.

One methodological caveat applies to these figures: GPU utilisation was sampled at 500 ms intervals, while each frame pair completes in approximately 115–128 ms. At most one in four frames falls within any given polling window, so mean utilisation values likely under-represent true average load. The peak values (88–100%) are less affected by this artefact and should be regarded as the more reliable indicator of burst resource pressure.

6.4 Limitations

Several limitations bound the scope of these findings. Because the pipeline is monocular, metric scale is unrecoverable from image pairs alone; all reported APE values reflect trajectory *shape* accuracy after Sim(3) alignment rather than absolute metric accuracy, and any deployment requiring metric output would need sensor fusion with a wheel encoder or depth source. A related concern is the synthetic domain gap: MambaGlue was trained on real-world datasets (MegaDepth, HPatches), and Gazebo’s phong-shaded textures differ substantially from real photographic images, so match confidence distributions and inlier rates may shift on a physical robot. The pipeline also operates as a pure open-loop integrator with no loop-closure mechanism, causing drift to accumulate monotonically — visible in the upward trend of all APE curves through the second half of the trajectory. On the computational side, all timing figures were collected on an RTX 4090; latency would be substantially higher on embedded platforms such as a Jetson Orin, and the relative ranking of architectures could shift depending on how each maps onto reduced memory bandwidth. Finally, and most critically for the accuracy claims, all comparisons rest on a single 4.8-minute trajectory with a non-deterministic RANSAC stage. The RMSE gap between the top two methods is only 0.201 m — small enough that bootstrapped confidence intervals across repeated runs would be needed to rule out a chance artefact. Complementing APE with Relative Pose Error at short time windows and logging per-method match-level statistics (inlier ratios, match counts) are the most direct paths to strengthening these conclusions in follow-on work.

7 Conclusion and Future Work

7.1 Conclusion

This paper presented a monocular visual odometry pipeline that replaces the conventional attention-based feature matcher with MambaGlue [3], a selective-state-space model designed for local feature correspondence. The pipeline couples SuperPoint [8] keypoint detection with MambaGlue matching and classical Essential Matrix pose recovery, and was evaluated against three established baselines — LightGlue, LightGlue with adaptive point pruning, and SuperGlue — on a simulated indoor trajectory of approximately 71.2 m driven over 4.8 minutes.

On the evaluated trajectory, MambaGlue produced the lowest APE RMSE of the four methods at 3.380 m after scale correction (4.75% of path length), compared to 3.581 m for LightGlue-Adaptive, 3.726 m for SuperGlue, and 3.750 m for LightGlue. The accuracy advantage came at a modest latency cost: MambaGlue added roughly 8–12 ms per frame relative to the fastest configuration (LightGlue), with an overall throughput of 7.80 FPS. Critically, all four methods operated well below the 25 FPS real-time threshold, with the common SuperPoint detector accounting for the majority of inference time — indicating that the detector, not the matcher, is the binding bottleneck for real-time deployment.

These results are preliminary. All accuracy figures derive from a single trajectory run with a stochastic pose estimator, and the inter-method RMSE differences are small enough that they could be influenced by run-to-run variance. Subject to those caveats, the evaluation is consistent with the hypothesis that SSM-based feature matching can serve as a viable alternative to attention-based and GNN-based matchers in monocular VO pipelines, without imposing a disproportionate computational penalty. Confirmation on multiple trajectories, real-world sensor data, and with match-level statistics would be needed to establish this more firmly.

7.2 Future Work

The accuracy rankings need verification across more trajectory runs; a single run leaves open how much variance comes from stochastic factors rather than the matcher itself. RPE at 1 s and 10 s windows would add a finer-grained view, and collecting inlier ratios and match count distributions per method would let you attribute differences to match quality directly.

SuperPoint is the bottleneck. Swapping it for DISK [4] or a pruned variant would likely close most of the gap to real-time without touching the matching architecture.

The pipeline has only been tested on Gazebo’s synthetic imagery, so it is unclear how much of MambaGlue’s accuracy advantage carries over to real photographs. KITTI or EuRoC would answer that directly.

Without loop closure, drift accumulates without bound. Adding a bag-of-words place recognizer and a pose graph back-end would cap long-term error regardless of trajectory length — probably a higher priority than the detection speedup if the goal is deployment.

On embedded hardware, profiling on a Jetson Orin comes before any other optimization. TorchScript export and half-precision inference are the most direct ways to cut latency there, though it is worth benchmarking both models separately to see where time actually goes.

Finally, monocular scale ambiguity is not a pipeline limitation but a geometric one: a single camera cannot recover metric scale. Stereo or RGBD fusion would give metric trajectories and make the APE numbers directly comparable to other benchmarks.

Acknowledgments

First and foremost, I thank my family for being the pillar of support for me throughout my life - I owe them a lifetime. I thank my advisor, Prof. Josiah P. Hanna, who was always there to guide my thoughts and ideas into structured research questions. I thank my friends and lab mates at the Prediction Action Lab of the University of Wisconsin-Madison for always motivating me to push beyond my limits and achieve the extraordinary.

References

- [1] A. Anilkumar. Visual odometry using ssm. <https://github.com/ashvin-a/Visual-Odometry-Using-SSM>, 2026.
- [2] M. O. Aqel, M. H. Marhaban, M. I. Saripan, and N. B. Ismail. Review of visual odometry: types, approaches, challenges, and applications. *SpringerPlus*, 5(1):1897, 2016.
- [3] K. Ryoo, H. Lim, and H. Myung. Mambaglu: Fast and robust local feature matching with mamba. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 5758–5765. IEEE, 2025.
- [4] M. Tyszkiewicz, P. Fua, and E. Trulls. Disk: Learning local features with policy gradient. *Advances in neural information processing systems*, 33:14254–14265, 2020.
- [5] G. Potje, F. Cadar, A. Araujo, R. Martins, and E. R. Nascimento. XFeat: Accelerated features for lightweight image matching. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [6] S. Wang, W. Li, Y. Wang, Z. Fan, Z. Huang, X. Cai, J. Zhao, and D. Li. Mambavo: Deep visual odometry based on sequential matching refinement and training smoothing. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 1252–1262, 2025.
- [7] M. A. Fischler and R. C. Bolles. Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, 24(6):381–395, 1981.
- [8] D. DeTone, T. Malisiewicz, and A. Rabinovich. Superpoint: Self-supervised interest point detection and description. In *Proceedings of the IEEE conference on computer vision and pattern recognition workshops*, pages 224–236, 2018.
- [9] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski. Orb: An efficient alternative to sift or surf. In *2011 International conference on computer vision*, pages 2564–2571. Ieee, 2011.
- [10] D. G. Lowe. Distinctive image features from scale-invariant keypoints. *International journal of computer vision*, 60(2):91–110, 2004.
- [11] P.-E. Sarlin, D. DeTone, T. Malisiewicz, and A. Rabinovich. Superglue: Learning feature matching with graph neural networks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 4938–4947, 2020.
- [12] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys. Lightglue: Local feature matching at light speed. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 17627–17638, 2023.